



Programming

Classes And OOP

Vassilis Markos
Mediterranean College

Week 09

Last Time's Exercises

Before That...

Take a moment to help me design our revision materials according to your needs:

<https://forms.gle/35JZhq9QD425xejs6>

Responses are anonymous so feel free to be as sincere as possible!

Last Time's Lab

Before we proceed with today's materials, let us complete any work undone on last time's lab. You can find it at:

`source/Programming_Lab_04.pdf`

Classes & Objects

Dogs

- Think about dogs for a moment. What are some common things all (or most) dogs do on a daily basis?
 - bark
 - walk
 - run
 - produce “[dog mess](#)”
- Write some python code that simulates the above!
- For instance, barking could be simulated by a message print on screen, walking / running could be simulated by a variable indicating a dog's location changing values etc.

Dogs

- Open `exm_01.py`.
- Read the code therein and try to understand how it works.
- Play around with the main function (have your dog bark, run, walk or make some mess in various combinations).
- Add any functionality you might find meaningful.
- Now, based on that, try to model a two-dog interaction. Things dogs do when encountering other canines include:
 - smelling each other
 - barking at each other
 - playing with each other
 - ...

Dogs

- Do you have any comments to make on your code? Is it easy to understand for others?
- What about having a dozen of dogs? What changes should you make for such a case to work properly?
- How about naming your dogs? (Alice, Bob, Charlie, David etc) How would you do it?
- What if you forget to name some dog? What should you do, then?
- Some dogs are more social than others. How can you take that into account?
- What dogs have to do with Python?

Classy Dogs

- Open `exm_02.py`.
- What do you see in there? Compare it to `exm_01.py`.
- What does this class thing mean?
- Don't look at me! Open a browser and try to understand yourselves!
- ...
- So, what is a **class**?
 - Briefly, classes are **abstractions** of (real) entities **incorporating relevant data and functionalities** that help us put things together in a meaningful way.

Classy Dogs

- What are some benefits of using classes?
 - We have everything in one place.
 - We avoid writing duplicate / similar lines of code.
 - We can bind functionality to data (this is important!)
 - We make our code easier to maintain (imagine having to change every single dog you have created to add a new feature...)
 - We can do some (almost) real magic (stay tuned).
- Extend the Dog class to allow them have a name (determined by users), and implement talking to, smelling, and playing with other dogs!

Classy Cats

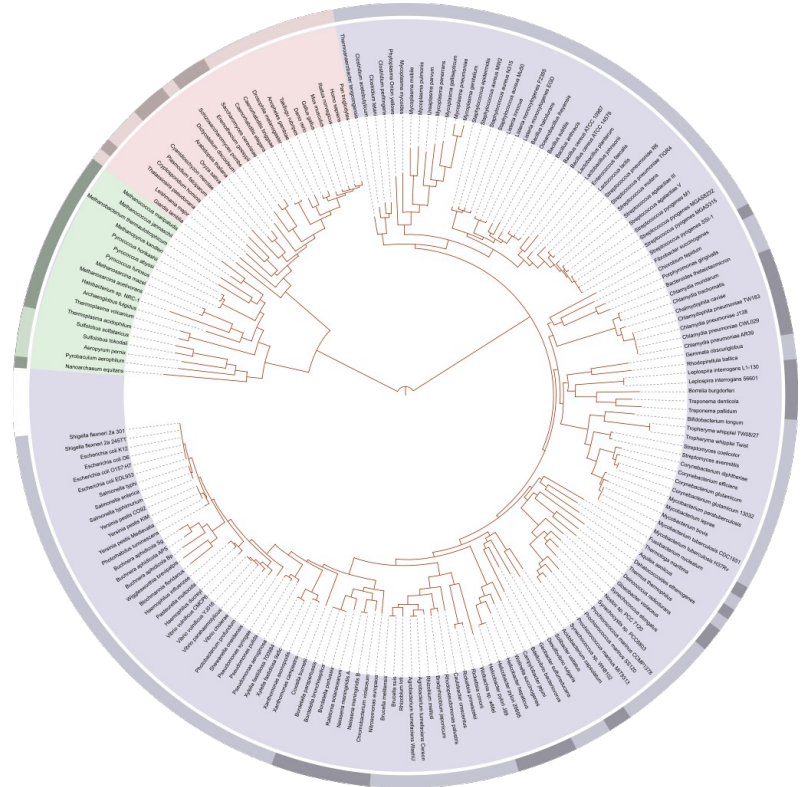
- What does a cat do as part of its daily routine?
 - It is adorable (well...)
 - It purrs
 - It changes its fur
 - It cleans itself
 - It plays with itself
 - ...
- Implement a python class named `Cat` that showcases some reasonable cat functionality.
- You can find a sample implementation at `exm_03.py`.

Object Oriented Programming (OOP)

- Using classes and all that stuff we have discussed so far is formally called **Object Oriented Programming**.
- The idea is, as we said, that we bind functionalities to data.
- A **class** in OOP is a description of an abstraction of some entity (e.g., the Dog and Cat classes we have designed so far).
- An **object** in OOP is a specific instance of a class. You can think of classes as recipes and of objects as the specific instances of a recipe (that bolognese you made yesterday).
- There are four main characteristics / benefits of OOP:
 - Inheritance
 - Data abstraction
 - Encapsulation
 - Polymorphism

Time for Biology

- This is a [phylogenetic tree](#).
- This is used to denote ancestry relationships between various species.
- As you move closer to this tree's root (here, in the centre of the graph) observe how you find common ancestors of several species.
- We do something similar in programming...



Inheritance

- In OOP we understand inheritance in a similar yet different way from biologists...
- Think of the Cat and Dog classes we have implemented so far. What do they have in common:
 - Both allow for self-playing as well as playing with others.
 - Both allow for moving objects (through functions and corresponding object fields).
 - Both allow for some sort of “talking”.
- However, Cats and Dogs also have their differences:
 - Dogs bark, cats purr (the also “meow”).
 - Dogs walk and run, while cats don’t (at least, so far).
 - Dogs produce mess, cats don’t.

Inheritance

- In OOP we tend to factor out common functionalities + data in what we call **Parent classes** (or **ancestor classes**, in general).
- In our case, we could design a `Pet` class, that contains all common functionalities and data fields of `Dogs` and `Cats`.
- Actually, “we”, means “you”, so play around a bit with python and write a `Pet` class, as described above!
- You can find a relevant implementation at `exm_04.py`.

Overwriting

- In `exm_04.py` observe the `Pet` and `Dog` classes.
- They both define a method named `talk()`. How does each work?
- Consider the following script and explain its behaviour:

```
if __name__ == "__main__":  
    alice = Pet("Alice")  
    bob = Dog("Bob")  
    alice.talk()  
    bob.talk()
```

Overwriting

- When inheriting a class we can redefine any method or field we want to. This is called **overwriting**.
- This is useful because, in many cases, there is a quasi-default behaviour for most child classes, but not for all. This is when overwriting shines.
- Some key takeaways about overwriting:
 - Overwriting a method in a child class **does not** change its parent's implementation.
 - An overwriting a method can still call its parent's very same method, if needed.

Polymorphism

What will this print? (exm_05.py)

```
from exm_04 import Pet, Dog, Cat

def main():
    alice = Dog("Alice")
    print(f"Is {alice.name} a dog? {isinstance(alice, Dog)}.")
    print(f"Is {alice.name} a cat? {isinstance(alice, Cat)}.")
    print(f"Is {alice.name} a pet? {isinstance(alice, Pet)}.")

if __name__ == "__main__":
    main()
```

Polymorphism

- **Polymorphism**: The situation where “a thing” is like “many things”.
- For instance, in the above:
 - Cats are also Pets.
 - Dogs are also Pets.
 - Dogs are not Cats or vice versa.
- How can we access a parent method from a child instance in python?
 - E.g., how can we make `alice` (a Dog) `talk()` like a Pet?
- Look around the web for that!
- The answer is... [drum roll]
 - We can't.
- This is important if it happens to know some other OOP language (e.g., Java).

Encapsulation & Data Abstraction

- **Encapsulation** [Take 1]: A fancy word to confuse you.
- **Encapsulation** [Take 2]: The act of binding functions / methods with data together.
 - E.g., a dog (datum) can bark (action → function).
- **Data Abstraction**: The act of keeping things at a higher level, hiding lower-level details.
- Revisit `exm_05.py` without taking a look at `exm_04.py`.
 - How are dog implemented in the background?
 - You don't know! (abstraction: the lower level is hidden from you)
 - How does class Cat implement `Cat.play()`?
 - You don't know! (abstraction: the lower level is hidden from you)
 - You can treat Pet, Dog and Cat as black boxes offering some functionality.

Class-ic Fun

- Do not execute the file below! (Uhm... Which file?)
- Open and study `exm_06.py`.
- What is going on in there? What will it print?
- Take your time, don't rush!
- Okay, now you can run the script.
- [break to get sufficiently surprised]
- What did happen?
- Can you explain it?

Class-ic Fun

bob:
bob.name
bob.location
<other bob stuff>
...
another_bob:
another_bob.name
<another_bob stuff>

What we have...

another_bob is just a different object living somewhere else in memory. For python objects, `a == b` just compares memory locations, so it yields **False!**

What we want...

bob:
bob.name
bob.location
<other bob stuff>
...
another_bob:
I am the same
as bob!

Python Magic Functions

- Python classes offer what is known as magic functions.
- Look around the web to find out what they are.
- [waiting for you to find out what they are]
- So, which function can help us make bob and another_bob the same?
- There is a popular magic function, `__eq__()`, which determines when two objects are the same.
- Try to use it to overwrite the default behaviour of `==` with python objects.

Class-ic Fun

bob:
bob.name
bob.location
<other bob stuff>
...
charlie
point to bob

What we have...

charlie = bob
just **creates a
reference** from
charlie to bob,
so any change on
either is reflected
on both!



What we want...

bob:
bob.name
bob.location
<other bob stuff>
...
charlie:
charlie.name
<other charlie stuff>

Homework

This is a self-study exercise on python classes. You have to follow the tutorial found below:

<https://realpython.com/python-classes/>

Observe all code found there and try it yourself (i.e., copy-paste it and run it locally). This will give you a really in depth starter on python classes.

Contact me for any questions at: v.markos@mc-class.gr

Fun Time!

Time for revision!

Start working on the exercises found in the 'source/portfolio' directory.

You can use any resources you want **wisely**!

Submit all your work on this Google Form once you are done (soft deadline: **Friday, 20/12/2024**):

<https://forms.gle/LSiNr4Rx5LNxUfjVA>

Any Questions?

Don't forget to fill-in the
questionnaire! (check right)



<https://forms.gle/zezKMVzhaT4xyJgk8>